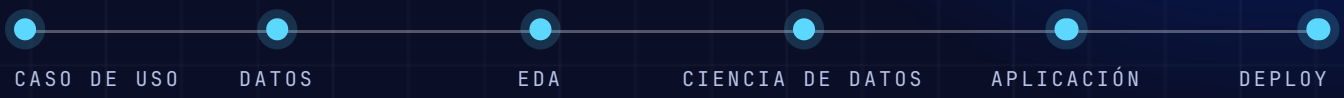
upgrade hub



PROYECTO FINAL

De los datos a una aplicación real

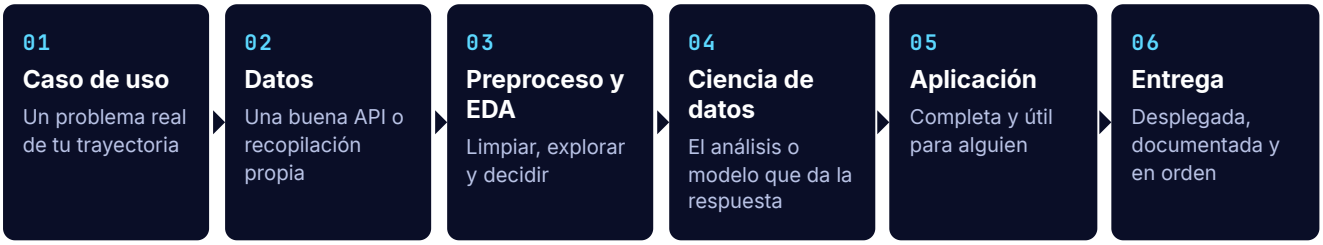
Un caso de uso de verdad, ligado a la trayectoria en datos que quieres desarrollar, llevado de principio a fin: datos, análisis, ciencia de datos y una aplicación desplegada y documentada.

EL ENCARGO

El proyecto, de un vistazo

El proyecto final consiste en resolver un **caso de uso real** con datos, de principio a fin: desde conseguir los datos hasta dejar funcionando una aplicación que alguien pueda usar. El tema lo eliges tú, y tiene que estar relacionado con la **trayectoria en datos que quieres desarrollar**.

El recorrido



Qué se pide

Parte	Requisito
Caso de uso	Real, y relacionado con la trayectoria en datos que quieres desarrollar.
Datos	Lo ideal es que procedan de una buena API o de un trabajo propio de investigación y recopilación.
Preprocesamiento y EDA	Los datos pasan por una fase de limpieza y otra de análisis exploratorio antes de usarse.
Aplicación	Completa, que solucione un problema y que integre ciencia de datos.
Panel de inteligencia SI PROCEDE	Aparte de la aplicación, cuando el caso lo necesite o como complemento.
Deploy	En Vercel, en Streamlit o en local. Si es en local, tiene que funcionar.
Documentación	El proyecto se entrega documentado.
Código	Comentado, ordenado y sin rastros evidentes de IA en comentarios ni textos.

> LA IDEA DE FONDO
>
> No se pide un cuaderno con gráficos ni un modelo suelto. Se pide un producto pequeño pero terminado: alguien que no eres tú lo abre, entiende para qué sirve y lo usa.

Cómo leer este documento

Las páginas siguientes desarrollan cada parte en el mismo orden del recorrido. La última incluye una lista de comprobación para repasar antes de entregar.

PASO 01

El caso de uso

Antes de buscar datos, decide qué problema vas a resolver y para quién. Todo lo demás (qué datos necesitas, qué analizas, qué construyes) sale de ahí.

Real
Alguien concreto tiene hoy ese problema: una empresa, un sector, una administración o tú mismo en tu trabajo.

De tu trayectoria
Se parece al trabajo que quieres hacer cuando acabes. Es lo que vas a enseñar cuando te pregunten qué sabes hacer.

Abarcable
Cabe en el tiempo que tienes. Mejor un problema acotado y bien resuelto que uno enorme a medias.

Según a qué te quieras dedicar

Todos los proyectos pasan por todas las fases. Lo que cambia con la trayectoria es dónde pones más profundidad.

Trayectoria	Dónde debería lucir el proyecto	Un ejemplo de enfoque
Análisis de datos y BI	EDA sólido, indicadores bien definidos y panel de inteligencia.	Evolución del precio del alquiler por barrio, con avisos cuando un barrio se desvía de su tendencia.
Ciencia de datos y ML	Modelado, validación e interpretación de resultados.	Previsión de demanda para un comercio, explicando qué factores pesan más.
Ingeniería de datos	Ingesta, automatización y calidad del dato.	Proceso que recoge datos de una API cada día, los valida y alimenta la aplicación.
IA aplicada	Modelos de lenguaje o de visión integrados en un producto.	Asistente que responde sobre documentación propia y mide la calidad de sus respuestas.

Cuatro preguntas para validar tu idea

- [ ] ¿Puedes explicar en dos frases qué problema resuelve y quién lo usaría?
- [ ] ¿Sabes ya de dónde van a salir los datos, y has comprobado que puedes acceder a ellos?
- [ ] ¿La respuesta necesita ciencia de datos, o bastaría con una hoja de cálculo?
- [ ] ¿Qué va a poder hacer el usuario con tu aplicación que antes no podía?

IDEAS QUE NO FUNCIONAN

- Proyectos sin problema: «voy a analizar un dataset de películas».
- Repetir un tutorial o un ejercicio de clase cambiando los datos.
- Elegir primero la técnica («quiero usar una red neuronal») y buscarle después un problema.

PASOS 02 Y 03

Los datos: origen, preprocesamiento y EDA

De dónde salen los datos

Lo ideal es que los datos lleguen por una de estas dos vías, o por una combinación de ambas.

Una buena API

- Está documentada y se mantiene.
- Da datos suficientes y actuales para tu problema.
- Sus condiciones de uso te permiten utilizarla.
- Dónde buscar: datos.gob.es, INE, AEMET OpenData, Eurostat o las APIs de tu sector.

Investigación y recopilación propia

- Reunir y cruzar varias fuentes que nadie ha juntado.
- Web scraping, respetando las condiciones de uso del sitio.
- Datos de una empresa u organización, con su permiso.
- Datos que generas tú: encuestas, registros, mediciones.

DATASETS YA PREPARADOS

Un CSV limpio descargado tal cual deja fuera buena parte del trabajo que se quiere ver. Si partes de uno, complétalo con otra fuente o actualízalo con datos recientes.

Deja constancia del origen: fuente, fecha de obtención y licencia de cada dato. Las claves de las APIs van en variables de entorno, nunca en el repositorio. Si hay datos personales, anonimízalos.

Preprocesamiento y EDA

Los datos no van directos a la aplicación. Antes pasan por dos fases que tienen que quedar visibles en la entrega, en cuadernos o en scripts.

Preprocesamiento

- Tipos y formatos: fechas, números, categorías.
- Valores nulos y duplicados.
- Valores atípicos y qué haces con ellos.
- Unión de fuentes y variables nuevas.
- Cada decisión, con su porqué.

Análisis exploratorio

- Parte de preguntas, no de gráficos.
- Distribuciones, relaciones y evolución en el tiempo.
- Gráficos legibles: título, ejes y unidades.
- Cada gráfico termina en una conclusión escrita.

QUÉ DISTINGUE UN BUEN EDA

No es una galería de gráficos. Cada análisis responde a una pregunta, y sus conclusiones explican por qué el modelo y la aplicación son como son.

PASOS 04 Y 05

La aplicación

Todo lo anterior desemboca en una aplicación completa: algo que una persona puede abrir y usar para resolver el problema del caso de uso.

Qué significa «completa»

- **Tiene un recorrido de principio a fin.** El usuario introduce datos o elige opciones y obtiene un resultado que le sirve.
- **Trabaja con los datos reales del proyecto**, no con ejemplos inventados para la demostración.
- **No se rompe a la primera.** Si falta un dato, la entrada no es válida o la API no responde, lo dice con un mensaje claro.
- **Se entiende sola.** Textos y resultados se interpretan sin que estés tú al lado.

Ciencia de datos dentro de la aplicación

La ciencia de datos tiene que estar integrada en la aplicación, no en un cuaderno aparte. Es la pieza que produce la respuesta que ve el usuario. Puede tomar muchas formas:

Tipo	Qué aporta a la aplicación
Predicción	Estima un valor o una categoría: un precio, un riesgo, una demanda.
Segmentación	Agrupa clientes, productos o zonas con comportamiento parecido.
Recomendación	Propone opciones a partir del historial o de la similitud entre elementos.
Series temporales	Proyecta la evolución de una magnitud y detecta cambios de tendencia.
Texto e imagen	Clasifica, resume o extrae información de documentos, reseñas o fotos.
Análisis estadístico	Compara grupos, mide efectos o señala anomalías.

Sea cual sea, explica cómo lo has evaluado (con qué métrica y sobre qué datos) y qué limitaciones tiene.

Panel de inteligencia SI PROCEDE

Si el caso de uso necesita seguir indicadores en el tiempo, o quieres completar la aplicación con una vista de análisis, añade un panel de inteligencia. Puede estar hecho en Power BI, Tableau o Looker Studio, o ser una sección de la propia aplicación.

UN BUEN PANEL

- Indicadores con una definición clara.
- Filtros que responden a preguntas reales.
- Una pantalla, una pregunta.

TEN EN CUENTA

El panel acompaña a la aplicación, no la sustituye. Si lo incluyes, enlázalo y explícalo en la documentación.

PASO 06

La entrega: deploy y documentación

Deploy

La aplicación se entrega desplegada por una de estas tres vías.

Opción	Cuándo encaja	Qué se entrega
Vercel	Aplicaciones web con interfaz propia y, si hace falta, una API por detrás.	URL pública y repositorio.
Streamlit	Aplicaciones de datos escritas en Python.	URL en Streamlit Community Cloud y repositorio.
Local	Proyectos que no se pueden publicar: modelos pesados que corren en tu equipo o datos privados.	Repositorio con instrucciones de arranque.

LOCAL VALE, PERO TIENE QUE FUNCIONAR

La aplicación debe arrancar en otro equipo siguiendo solo el README. Para comprobarlo:

- Dependencias con versiones en `requirements.txt` o `package.json`.
- Un `.env.example` con las variables necesarias, sin valores reales.
- Los datos incluidos, o un script que los descarga.
- Un único comando de arranque, escrito en el README.
- Probado en un entorno limpio: clona el repositorio en otra carpeta, crea un entorno nuevo y sigue tus propias instrucciones.

Documentación

El `README.md` es la puerta de entrada. Como mínimo cuenta:

1. Qué problema resuelve y para quién.
2. Los datos: origen y cómo se obtienen.
3. Preprocesamiento y EDA: decisiones y hallazgos principales.
4. Ciencia de datos: enfoque, métricas y limitaciones.
5. La aplicación: cómo se usa, con capturas, y enlace al deploy.
6. Instalación y ejecución, paso a paso.
7. Estructura del repositorio.
8. Qué mejorarías con más tiempo.

Una estructura de repositorio posible, que puedes adaptar a tu proyecto:

```
proyecto-final/
├── README.md
├── requirements.txt
├── .env.example
├── data/          bruto y procesado
├── notebooks/     01_preproceso
│                  02_eda · 03_modelo
├── src/           funciones comunes
├── app/           la aplicación
└── docs/          capturas y panel
```

ANTES DE ENTREGAR

Código en orden y sin rastros de IA

Código comentado y ordenado

- Nombres de variables y funciones que se entienden.
- Comentarios que explican el porqué de una decisión, no lo que ya dice la línea.
- Sin código muerto, pruebas a medias ni celdas vacías.
- Los cuadernos se ejecutan de arriba abajo sin errores.
- Sin rutas de tu ordenador ni claves escritas en el código.

Lo que entregas es tuyo

Puedes apoyarte en asistentes de IA para trabajar. Lo que no puedes es entregar algo que no has leído.

Todo lo que aparece en el repositorio lo firmas tú: tienes que entenderlo, haberlo revisado y poder explicarlo si se te pregunta.

RASTROS QUE NO PUEDEN APARECER

- **Prompts pegados** en comentarios, celdas o textos: `# hazme una función que limpie los nulos y devuelva el dataframe`
- **Respuestas literales del asistente:** «¡Claro! Aquí tienes el código actualizado», «Espero que te sirva», «Como modelo de lenguaje...».
- **Comentarios que cuentan la conversación:** `# Corregido según lo solicitado`, `# Versión mejorada`, `# Aquí está el cambio que pediste`.
- **Comentarios de relleno** en cada línea: `# Importamos pandas`, `# Creamos el modelo`.
- **Textos genéricos** en el README o en la aplicación que podrían ser de cualquier proyecto, o que describen cosas que el tuyo no hace.

Repaso rápido: lee el repositorio entero de arriba abajo como si fuera de otra persona, usa el buscador del editor para localizar frases como las anteriores y reescribe con tus palabras lo que no suene a ti.

Lista de comprobación

- [ ] El caso de uso es real y encaja con mi trayectoria.
- [ ] El origen de los datos está documentado.
- [ ] Preprocesamiento y EDA están en la entrega, con decisiones y conclusiones.
- [ ] La aplicación resuelve el problema e integra ciencia de datos.
- [ ] El panel de inteligencia, si lo hay, está enlazado y explicado.
- [ ] El deploy funciona: la URL abre, o el arranque en local está probado en limpio.
- [ ] El README cubre los ocho puntos.
- [ ] El código está comentado y ordenado.
- [ ] No hay claves ni rutas personales en el repositorio.
- [ ] No quedan prompts ni respuestas literales de IA.